

SD Basic - Math Functions

Arithmetic, division and remainders, rounding, powers, trigonometry and random numbers.

This page covers the numeric side of SD BASIC: the arithmetic operators, the functions that divide and round, powers and logarithms, trigonometry, the functions that reduce a dynamic array to a number, bit manipulation, and the control over how many decimal places a number shows when it becomes a string.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. Braces mark optional parts of a syntax line; in the tables, *italics* mark something you supply and **bold** marks a word typed as it stands.

Where this port behaves differently from the older OpenQM and SD documentation, the difference is called out rather than left for you to discover.

Numbers in SD BASIC

SD holds a number as either an **integer** or a **floating-point** value and moves between them as needed. You do not declare which; an expression that can stay integral does, and one that cannot becomes floating point.

Everything in SD is ultimately a string, so a variable holding `"12"` is a number wherever a number is wanted. A string that is not numeric evaluates as zero in an arithmetic context, and `num()` below is how you test before relying on that.

ARITHMETIC OPERATORS

OPERATOR	MEANING
<code>+</code>	add
<code>-</code>	subtract, and unary minus
<code>*</code>	multiply
<code>/</code>	divide — the result may be fractional
<code>**</code> . <code>^</code>	raise to a power. The two spellings are the same operator

There are also compound assignment operators — `+=`, `-=`, `*=`, `/=` and `:=` for concatenation — so `total += n` is `total = total + n`.

There is no integer-divide operator. `/` always divides fully:

```
print 7 / 2      ;* prints 3.5
```

For a whole-number result use `div()`, `idiv()` or `rdiv()`, described below. `//` is **not** integer division — it is the marker for an unnamed common block, as in `common // a, b, c`.

PRECEDENCE

Operators lower in this table are applied first. Anything in parentheses is evaluated before the expression around it.

OPERATORS

- | | |
|---|---|
| 1 | <code>**</code> <code>^</code> |
| 2 | <code>*</code> <code>/</code> |
| 3 | <code>+</code> <code>-</code> |
| 4 | the <code>fmt</code> operator |
| 5 | <code>:</code> concatenation |
| 6 | <code><</code> <code>></code> <code>=</code> <code>#</code> <code><=</code> <code>>=</code> <code><></code> <code>matches</code> , and their word forms <code>lt</code> <code>gt</code> <code>eq</code> <code>ne</code> <code>le</code> <code>ge</code> |
| 7 | <code>and</code> <code>&</code> <code>or</code> <code>!</code> |

`and` and `or` have the same precedence, which is not what most languages do. They are applied left to right as they are met, so `a or b and c` is `(a or b) and c` — not `a or (b and c)`. **Parenthesise any condition that mixes them.** This has been true since OpenQM and is not a change made by this port, but it surprises people arriving from C, Python or SQL, where `and` binds tighter.

DIVIDING BY ZERO

Dividing by zero stops the program with a runtime error. That can be turned into a warning that yields zero and carries on:

```
option div.zero.warning
```

The option applies to `/`, `div()`, `idiv()` and `rdiv()` alike.

Dividing to a whole number

Three functions divide and return an integer, and **they do not round the same way**. Picking the wrong one is the commonest numeric bug in MultiValue code, so the differences are set out in full.

FORMAT

```
div(dividend, divisor)
idiv(dividend, divisor)
rdiv(dividend, divisor)
```

FUNCTION	RULE
div()	divides and discards the fraction, always towards zero
idiv()	towards zero when both arguments are integers, downwards when either is floating point — see the warning below
rdiv()	divides and rounds to the nearest integer, with a half rounded away from zero

THE FOUR SIDE BY SIDE

EXPRESSION	RESULT	EXPRESSION	RESULT
div(7, 2)	3	rdiv(7, 2)	4
div(-7, 2)	-3	rdiv(-7, 2)	-4
div(7, -2)	-3	rdiv(7, -2)	-4
div(-7.5, 2)	-3	rdiv(5, 2)	3
idiv(7, 2)	3	rdiv(-5, 2)	-3
idiv(-7, 2)	-3		

`idiv()` **changes its rounding when a floating-point number is involved, and nothing in the call shows it.**

With two integers it truncates towards zero; if either argument is floating point it takes the floor, which for a negative result is a different number:

```
idiv(-7, 2) ;* -3 both arguments are integers idiv(-7.0, 2) ;* -4 one argument is floating point
```

The two differ only for negative results — for positive ones truncation and floor agree. **If the sign can be negative, use `div()`**, which truncates towards zero whatever the argument types are. This behaviour is inherited from OpenQM and has been kept for compatibility; it is not new in this port.

`div()` returns a whole number even when the arguments are fractional: `div(7.5, 2)` is `3`.

Remainders

```
mod(dividend, divisor)
rem(dividend, divisor)
```

Both give the remainder of a division and **they differ on the sign**. `mod()` takes the sign of the *divisor*; `rem()` takes the sign of the *dividend*.

EXPRESSION	MOD	REM
<code>(7, 3)</code>	<code>1</code>	<code>1</code>
<code>(-7, 3)</code>	<code>2</code>	<code>-1</code>
<code>(7, -3)</code>	<code>-2</code>	<code>1</code>
<code>(-7, -3)</code>	<code>-1</code>	<code>-1</code>

`mod()` is the mathematical modulus — the result always lies between zero and the divisor — which is what you want for wrapping a value into a range, such as an index into a cyclic table. `rem()` is the remainder C and most other languages give, and is what you want when you are reconstructing a value from a quotient and a remainder.

Neither raises a divide-by-zero error. A divisor of zero returns the dividend unchanged: `mod(7, 0)` and `rem(7, 0)` are both `7`. A loop that relies on a remainder shrinking will not terminate if the divisor can reach zero, and nothing will tell you.

Truncation, sign and magnitude

```
int(expr)
abs(expr)
neg(expr)
```

`int()` discards the fractional part, **towards zero**. `int(3.7)` is `3` and `int(-3.7)` is `-3`. It does not round: use `rdiv(n, 1)` or add `0.5` if you want rounding

`abs()` the magnitude, with any sign removed. `abs(-3.5)` is `3.5`

`neg()` the value with its sign reversed. `neg(3)` is `-3`

`int()` is affected by a configuration parameter. `INTPREC` sets how many decimal places are considered before truncating, which stops a value that is `2.999999999` only because of floating-point representation from truncating to `2`. It defaults to `13` and may be set from `0` to `14`. `config('INTPREC')` reports the value in force, and `config gp1` is not where to look — the whole parameter list is shown by the `config` verb.

Powers, roots and logarithms

```
pwr(base, exponent)
sqrt(expr)
exp(expr)
ln(expr)
```

`pwr()` *base* raised to *exponent*. Identical to the `**` and `^` operators, so `pwr(2, 10)`, `2 ** 10` and `2 ^ 10` all give `1024`. A fractional exponent works: `pwr(2, 0.5)` is `1.4142`

`sqrt()` the square root. `sqrt(2)` is `1.4142`

`exp()` *e* raised to the given power. `exp(1)` is `2.7183`

`ln()` the natural logarithm, base *e*. `ln(10)` is `2.3026`

There is no base-10 logarithm function. Divide by the natural log of ten: `ln(x) / ln(10)`.

`sqrt()` and `ln()` stop the program on a negative argument — "*SQRT() attempted for negative value*" and "*LN() attempted for negative value*". Neither returns an error code you can test, so guard the argument yourself if it can go negative.

The four decimal places shown above are not the precision of the calculation — they are the default precision applied when the number is turned into a string for printing. See *Controlling how numbers print*.

Trigonometry

```
sin(expr)    asin(expr)
cos(expr)    acos(expr)
tan(expr)    atan(expr)
```

The arguments and results are in degrees, not radians. This catches people out constantly, because almost every other language's maths library uses radians. There is no radian variant.

EXPRESSION	RESULT	EXPRESSION	RESULT
<code>sin(30)</code>	<code>0.5</code>	<code>asin(0.5)</code>	<code>30</code>
<code>cos(60)</code>	<code>0.5</code>	<code>acos(0.5)</code>	<code>60</code>
<code>tan(45)</code>	<code>1</code>	<code>atan(1)</code>	<code>45</code>

To work in radians, convert: multiply by `180 / 3.14159265358979` on the way in, and divide on the way out.

Random numbers

```
rnd(limit)
randomize {seed}
```

`rnd()` returns a whole number from `0` to *limit* minus one — so `rnd(10)` gives `0` through `9`, confirmed over 500 draws. A negative *limit* gives a negative result in the mirrored range: `rnd(-10)` returns `0` down to `-9`.

`randomize` seeds the generator. With a *seed* the sequence repeats identically every run — `randomize 42` followed by three `rnd(1000)` calls gives the same three numbers every time — which is what you want for a reproducible test. With no argument it seeds from the clock, which is what you want in production.

```
randomize 42
for i = 1 to 5
    print rnd(100)
next i
```

This is not a cryptographic random number generator and must not be used to make passwords, tokens or keys. It is the C library's `rand()`, seeded from a 32-bit value. For anything security-bearing use `sdencrypt()` and the facilities described under *Security* rather than building your own.

Continued in

SD Basic - Numeric Tests and Formatting — reducing an array to a number, numeric tests, bit operations, checksums and how numbers print.